Assignment #6 — Visitor Pattern: Advanced File System

Student Begaliev Bakdaulet | **Group** SE-2526

Course ShP-2216, Software Design Patterns | **Language** Java 17 / Maven

Academic year 2026–2027 | **Repository** github.com/bakdauletbegaliev/visitor

1. Purpose and introduction

This project demonstrates the **Visitor behavioral pattern** on a small in-memory file system. A tree contains folders (**DirectoryNode**) and files (**FileNode**). Three independent visitors calculate the total size, search for files, and collect statistics. No actual disk files are created.

2. Architecture and double dispatch

Element defines **accept(Visitor)**, **getName()** and **getChildren()**. **AbstractNode** holds the node name and its parent. **FileSystemWalker** is the single traversal owner. **Visitor** has overloaded **visit(FileNode)** and **visit(DirectoryNode)** methods. The complete UML class diagram is on page 4.

```
FileNode.accept(visitor)      -> visitor.visit(this)
DirectoryNode.accept(visitor) -> visitor.visit(this)
```

First Java selects the concrete node's **accept** implementation; next it selects the matching **visit** overload. This is double dispatch. The walker does not use **instanceof**, casts, or reflection.

3. Demonstration: chosen test data

```
project/
|-- notes.txt (120 B)
|-- src/
|   |-- Main.java (2400 B)
|   `-- utils/
|       `-- Helper.java (800 B)
|-- assets/
|   `-- logo.png (1500 B)
`-- empty/
```

The search uses a case-insensitive filename fragment **.java** and inclusive minimum size **800 bytes**. The path of the selected root directory is omitted from matches.

```
Total size: 4820 bytes
Java files (at least 800 bytes): [src/Main.java, src/utils/Helper.java]
Files: 4
Directories: 5
Maximum depth: 3
```

Design choices & Clean Code

4. Five concrete Clean Code practices

Practice	Where it is used	Why it improves the code
Meaningful naming	TotalSizeVisitor, SearchVisitor, getTotalBytes()	The names say what each operation does without guessing.
Single Responsibility	FileSystemWalker.walkNode(); visitor classes	Traversal happens in one place; every visitor handles one operation.
Small methods	FileNode.accept(); SearchVisitor.visit(FileNode)	Each method does one short, understandable action.
Encapsulation	DirectoryNode.addChild() / getChildren()	Only validated insertions are allowed; callers cannot change the child list.
No type-check chains	accept() and Visitor.visit() overloads	Polymorphism selects file or directory behavior instead of instanceof/if chains.

5. Traversal, paths and visitor state

The **walker** owns depth-first traversal. It calls **setContext(relativePath, depth)**, then **node.accept(visitor)**, then recursively walks each child. A leaf returns an empty child list. So every reachable element is visited once per operation, including empty folders. The selected root is at **depth 0**; the deepest visited element determines maximum depth.

Relative paths use **/**. For a folder root, child names form the path (e.g., **src/utils/Helper.java**); for a file root, the result is the filename. **SearchVisitor.getMatches()** sorts matches lexicographically so the result does not depend on insertion order.

The walker calls **visitor.reset()** at the beginning of each run. Thus the three visitor instances are **reusable**; repeated runs do not multiply counts or keep old search results. The total-size visitor uses a **long** value and detects overflow with **Math.addExact()**.

6. Tree integrity and extension boundaries

All node names are fixed on creation. **FileNode** rejects negative sizes. **DirectoryNode.addChild()** rejects null children, duplicate sibling names, multiple parents, adding itself, and attaching an ancestor below its descendant. **getChildren()** returns an unmodifiable snapshot. Valid deeper nesting is supported.

Adding a new operation: implement another Visitor and call the walker; existing node classes and their accept methods stay unchanged. **Adding a new element type:** expand the Visitor interface and update its implementations, which is the main trade-off of Visitor. The commit plan places StatisticsVisitor after the first two visitors to demonstrate that extension.

Testing, results & conclusion

7. Automated verification (JUnit 5)

Three JUnit test classes: **VisitorDispatchTest** (2 tests), **VisitorOperationsTest** (11 tests), **TreeIntegrityTest** (9 tests). Together they contain **22 test methods**, mapping to all eight required checkpoints.

ID	Verification target	Test class
T1	Correct overloaded visit() for file and directory	VisitorDispatchTest
T2	Size across several nesting levels	VisitorOperationsTest
T3	Filename criterion AND inclusive size threshold	VisitorOperationsTest
T4	Sorted root-relative search paths with / separator	VisitorOperationsTest
T5	File count, directory count, maximum depth	VisitorOperationsTest
T6	Empty directory, file root, no matches	VisitorOperationsTest
T7	Reset on repeated runs and new roots	VisitorOperationsTest
T8	Reject invalid mutations; support valid children	TreeIntegrityTest

Result available in this build environment: the Java 17 source and all 22 test methods compiled, and a local JUnit-compatible assertion runner executed **22 passed, 0 failed**. Maven was not installed here; an official JUnit 5 run using **mvn test** is still needed on the student computer before submission.

8. Defense demonstration

Test: mvn test Run: mvn -q compile exec:java. In **Main.java**, change the minimum search size from **800** to **1000**; now only **src/Main.java** matches. Restore 800 to show that 800 is included. For the dispatch trace, open **FileNode.accept()** and **DirectoryNode.accept()**, then the two **Visitor.visit()** declarations.

9. Conclusion and repository

Visitor keeps the object structure simple while allowing different operations on one tree. Traversal is centralized, state is reset, and the tree blocks invalid changes. The cost is several small classes and extra effort when adding a new element type.

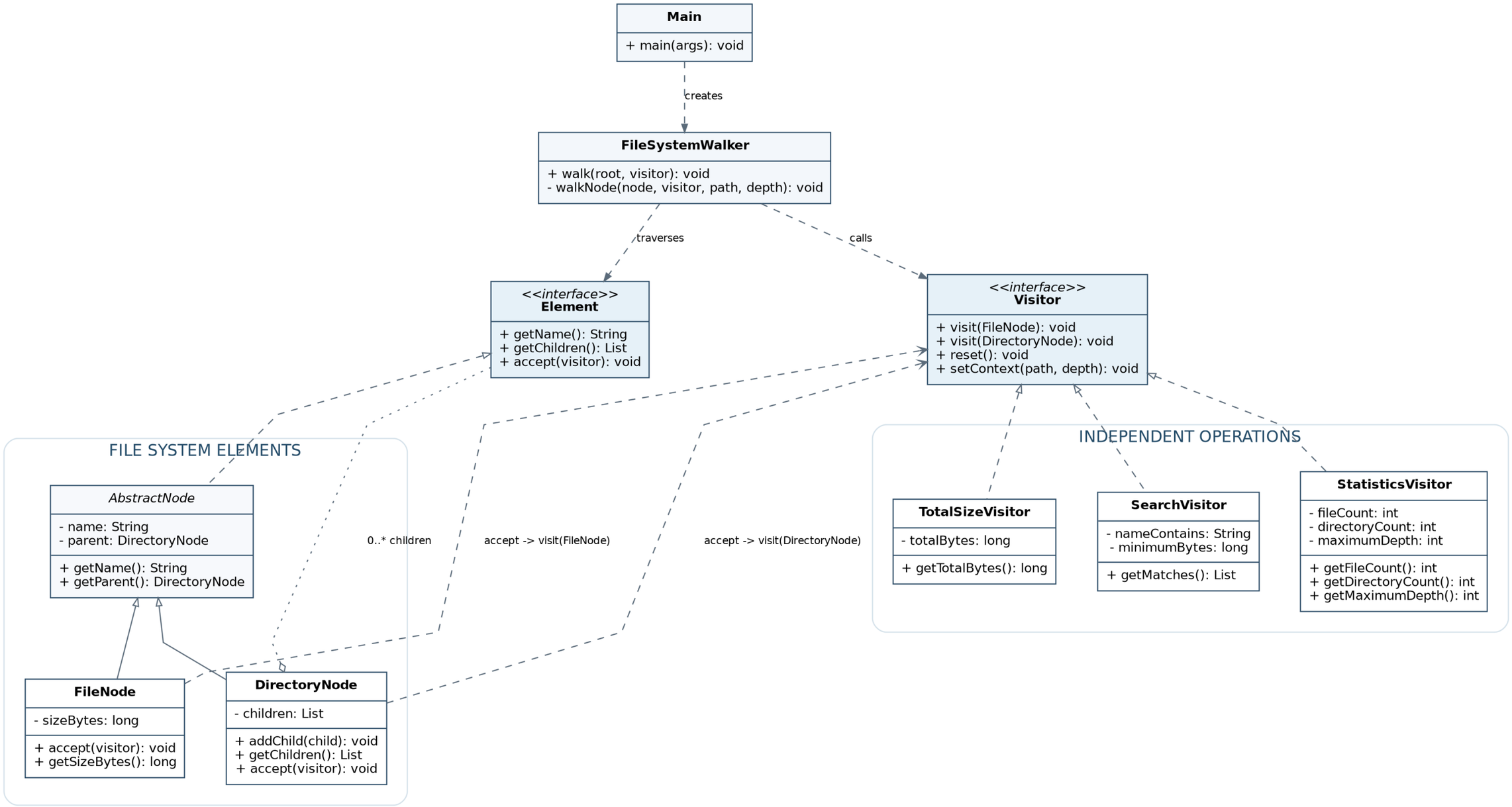
GitHub: https://github.com/bakdauletbegaliev/visitor

References and disclosure

Astana IT University, Assignment 06 — Visitor Pattern: Advanced File System (provided brief).
Freeman & Robson, Head First Design Patterns, 2nd edition (Visitor).
Refactoring.Guru, Visitor: https://refactoring.guru/design-patterns/visitor
Drafting support: OpenAI ChatGPT; review and independently verify the work under course rules.

UML Class Diagram

Visitor Pattern - Advanced File System | Figure 1



Dashed hollow triangles: interface implementation; solid hollow triangles: inheritance; dotted diamond: directory holds children.